

Tiempo de evento, ventanas y marcas de agua

Streaming de datos y sus aplicaciones

Maestría en Inteligencia Artificial

Facultad Politécnica - Universidad Nacional de Asunción

Rodrigo Parra, M.Sc.

Clase 4

Contents

1. El problema del tiempo

- La clase anterior resolvió la repetición, no el orden
- Una compra ocurrida a las 09:59 llega a las 10:03
- Hoy responderemos cuatro preguntas
- Resultados de aprendizaje
- El caso conductor: movilidad urbana

2. Tres relojes

- Un mismo evento tiene varias coordenadas temporales

- El tiempo de evento preserva el significado del dominio
- El tiempo de procesamiento también puede ser correcto
- Entre ambos relojes aparece el skew
- ¿De dónde sale el retraso variable?
- Actividad: identifica el reloj correcto

3. Ventanas: dónde calcular

- Una agregación global sobre un flujo nunca termina
- La ventana responde dónde vive el resultado
- Ventanas fijas: particiones regulares y no solapadas
- Ventanas deslizantes: una vista móvil del pasado reciente
- Ventanas de sesión: el usuario define los límites

- La ventana global no elimina la necesidad de emitir
- Elegir ventana es una decisión semántica
- Los bordes pueden cambiar la historia
- Actividad: elige la ventana

4. Watermarks: progreso en tiempo de evento

- Una ventana no sabe por sí sola cuándo está completa
- La marca de agua es una afirmación de progreso
- El watermark avanza cuando avanzan las fuentes
- Watermark perfecto: conocimiento completo
- Watermark heurístico: una estimación útil y falible
- Evento temprano, a tiempo y tardío son relaciones

- Una marca de agua no es un SLA
- Actividad: mueve la marca de agua

5. Triggers: cuándo materializar

- Un trigger produce una versión observable: un pane
- Emisiones tempranas compran latencia con provisionalidad
- La emisión on-time usa el progreso estimado
- Emisiones tardías corrigen después del watermark
- Los triggers pueden observar señales distintas
- Acumular define cómo leer varias emisiones
- Discarding sirve cuando downstream acumula deltas
- Accumulating encaja con upsert por ventana

- Retractions ayudan cuando el resultado cambia de lugar
- Un patrón frecuente combina tres momentos

6. Datos tardíos y cierre

- Allowed lateness define cuánto tiempo corregimos
- Tardío no significa aceptado indefinidamente
- Más lateness compra completitud con recursos
- Mide la distribución, no solo el promedio
- El contrato de salida debe expresar revisiones
- El triángulo de diseño no desaparece

7. Diseño de una política completa

- Caso: retraso medio por línea cada cinco minutos

- Decisión 1: tiempo de evento del dispositivo
- Decisión 2: ventana fija de cinco minutos
- Decisión 3: early, on-time y late
- Decisión 4: acumular y sobrescribir
- Decisión 5: lateness de quince minutos
- Pseudocódigo conceptual en Apache Beam
- Pruebas que demuestran la política
- Checklist de revisión

8. Síntesis y aplicación

- Tarea 2: razonar sobre datos desordenados
- Una buena entrega muestra la evolución del resultado

- Errores frecuentes que debes evitar
- Las cuatro preguntas forman un contrato
- Cinco ideas para llevar
- Lecturas para consolidar
- Puente hacia la clase 5
- Fuentes y atribuciones

El problema del tiempo

Clase 3

- reintentos y duplicados
- commits y checkpoints
- idempotencia y deduplicación
- efectos observables bajo fallos

Clase 4

- eventos que llegan tarde
- orden de llegada distinto al orden real
- resultados parciales
- decisiones sobre cuándo publicar

Idea clave

Un evento puede procesarse una sola vez y aun así terminar en la agregación equivocada si usamos el reloj incorrecto.

Evento	Ocurrió	Llegó	Monto
sale-184	09:59:52	10:03:11	₡ 850.000

Si agrupamos por llegada

La venta aparece en la hora
10:00–11:00.

Si agrupamos por ocurrencia

La venta pertenece a la hora
09:00–10:00.

¿Qué resultado representa mejor al negocio?

Qué calculamos

Transformaciones: suma, promedio, conteo, máximo, modelo.

Dónde en tiempo de evento

Ventanas: fijas, deslizantes, sesiones o global.

Cuándo publicamos

Watermarks y triggers: temprano, a tiempo y tarde.

Cómo se relacionan las revisiones

Acumulación: deltas, reemplazos o retractaciones.

Al final de la clase deberías poder

- Diferenciar tiempo de evento, ingesta y procesamiento.
- Elegir ventanas fijas, deslizantes o de sesión según el significado analítico.
- Explicar qué afirma —y qué no afirma— una marca de agua.
- Interpretar panes tempranos, a tiempo y tardíos.
- Definir lateness permitido y modo de acumulación.
- Justificar el trade-off entre latencia, completitud y costo.

Pregunta guía: ¿cuándo sabemos lo suficiente para emitir un resultado útil?

Eventos de viajes

Cada vehículo emite `trip_update` con `vehicle_id`, `route_id`, `event_time`, posición y retraso estimado.

Pregunta

¿Cuál es el retraso medio por línea?

Frecuencia

Actualización visible cada 30 segundos.

Realidad

Algunos vehículos quedan offline varios minutos.

Idea clave

La métrica necesita rapidez para operar y corrección temporal para comparar rutas.

Tres relojes

Tiempo de evento

Reloj: reloj del dominio

Responde: ¿cuándo ocurrió el hecho?

Tiempo de ingesta

Reloj: entrada a la plataforma

Responde: ¿cuándo lo recibió el sistema?

Tiempo de procesamiento

Reloj: reloj del operador

Responde: ¿cuándo se ejecutó el cálculo?

Ejemplo

Un vehículo mide a las 10:00:05, Kafka recibe a las 10:02:40 y el agregador procesa a las 10:02:43.

Preguntas de negocio

- ventas por hora comercial
- sesiones de navegación
- retraso de un viaje
- consumo facturable
- anomalías sobre una señal física

Lo que necesitan

Agrupar según cuándo ocurrió el hecho, aunque se observe más tarde.

Idea clave

Si un replay cambia el resultado solo porque se ejecutó otro día, probablemente usamos tiempo de procesamiento donde necesitábamos tiempo de evento.

Preguntas operativas

- ¿cuántos eventos procesa el sistema por segundo?
- ¿cuánta carga recibió el servicio en los últimos 30 segundos?
- ¿cuánto tardó el pipeline en ejecutar una etapa?
- ¿cuándo debemos actualizar un dashboard provisional?

Tiempo de evento

Describe el mundo que modelamos.

Tiempo de procesamiento

Describe el comportamiento del sistema.

La elección depende de la pregunta, no de cuál reloj sea más fácil de leer.

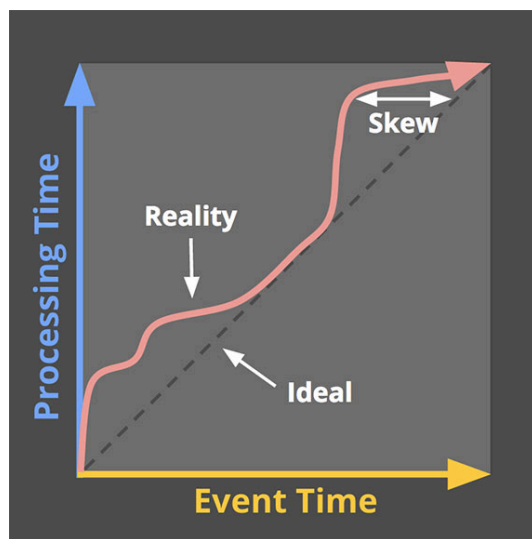


Figura: Tyler Akidau, [Streaming 101](#).

Lectura

- la diagonal representa observación inmediata
- la curva real se retrasa de forma variable
- la distancia entre ambas cambia con la red, carga y fuente
- no existe un retraso universal que funcione siempre

Idea clave

El desorden es una propiedad normal de un sistema distribuido, no una excepción.

Fuente

- móviles offline
- sensores con buffer
- relojes desajustados
- lotes desde dispositivos

Transporte

- congestión
- particiones de red
- reintentos
- múltiples regiones

Procesamiento

- backpressure
- hot keys
- pausas y reinicios
- prioridades distintas

La llegada ordenada requiere coordinación; el desorden no.

Pregunta	Evento	Ingesta	Proceso
Facturación por hora de consumo	✓		
QPS observado por el gateway		✓	
Duración de una transformación			✓
Sesión de uso de una app móvil	✓		
Alerta por atraso del pipeline			✓

Discusión

¿Qué cambiaría durante un replay histórico? Esa pregunta suele revelar el reloj apropiado.

Ventanas: dónde calcular

Consulta aparentemente simple

```
SUM(amount) GROUP BY route_id
```

Datos acotados

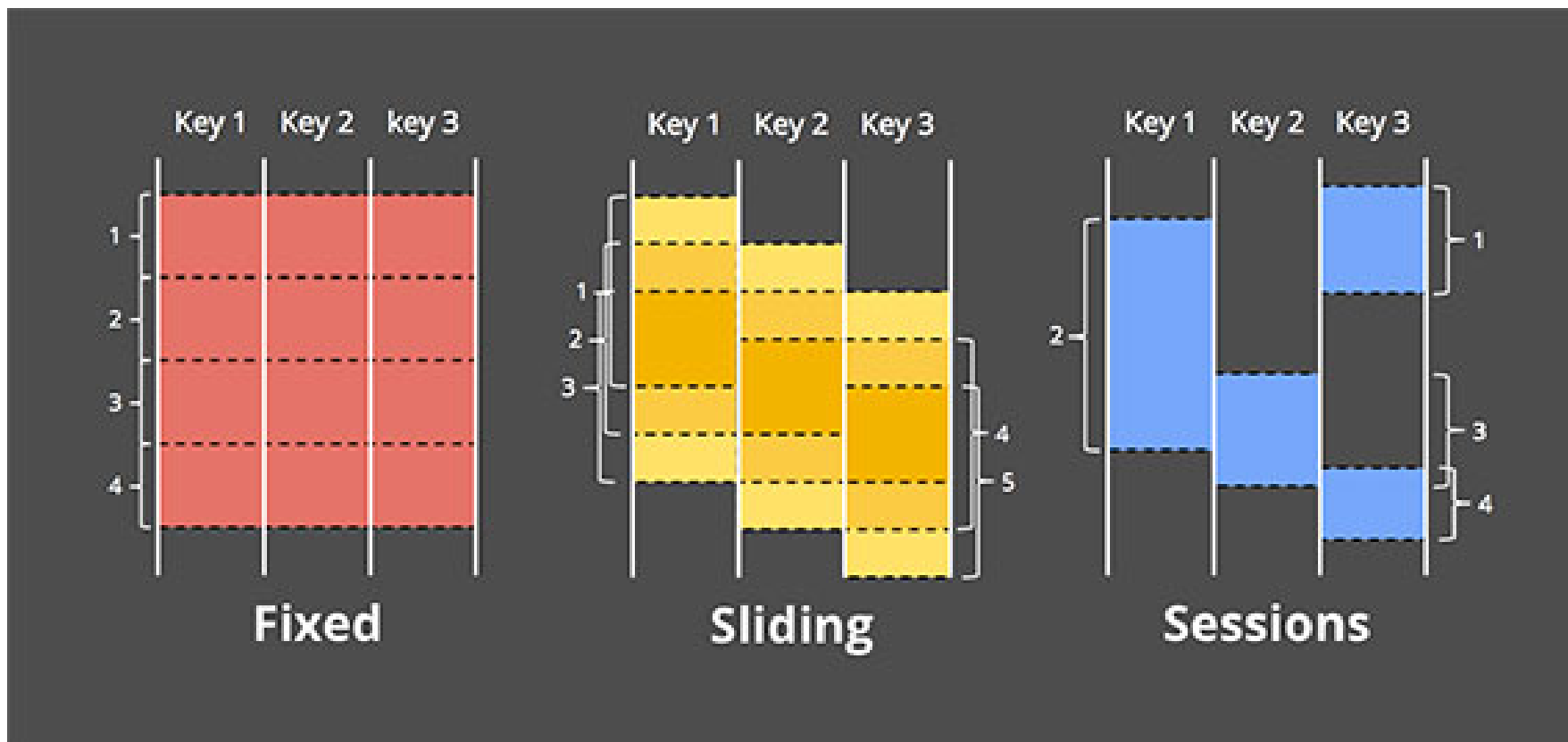
La entrada termina; la suma final puede materializarse.

Datos no acotados

Siempre puede llegar otro evento; “final” no existe sin una política.

Idea clave

Una ventana asigna cada evento a un conjunto finito de tiempo sobre el que podemos mantener estado y emitir resultados.



Importante

Ventana y trigger son decisiones distintas: la ventana agrupa en tiempo de evento; el trigger decide cuándo observar una versión del resultado.

[10:00, 10:05)	[10:05, 10:10)	[10:10, 10:15)	[10:15, 10:20)
----------------	----------------	----------------	----------------

Útiles para

- ventas por hora
- alertas por minuto
- SLA por día
- conteos operativos

Decisiones

- duración
- alineación y zona horaria
- timestamp usado
- tratamiento de bordes

Convención común: intervalos semiabiertos $[\text{inicio}, \text{fin})$; un evento exactamente en fin pertenece a la ventana siguiente.

Configuración

Tamaño: 10 minutos · **Período:** 2 minutos

Emisión	Rango de tiempo de evento
10:10	[10:00, 10:10)
10:12	[10:02, 10:12)
10:14	[10:04, 10:14)
10:16	[10:06, 10:16)

Idea clave

Un evento puede pertenecer a varias ventanas; eso aumenta el costo de estado y agregación.

Regla

Agrupar eventos por clave mientras la separación entre eventos sea menor que un gap de inactividad.

09:00 click	09:04 search	09:11 buy	10:02 open	10:08 close
-------------	--------------	-----------	------------	-------------

Sesión 1

09:00–09:11; los gaps no superan 15 min.

Sesión 2

10:02–10:08; 51 min separan ambas sesiones.

Los eventos tardíos pueden unir dos sesiones que antes parecían separadas.

Ventana global

Todos los eventos pertenecen a una única ventana conceptual.

Puede servir para

- estado acumulado por clave
- máximos históricos
- modelos incrementales
- monitoreo desde el inicio

Pero necesita

- triggers periódicos
- política de retención
- estado acotado o compactado
- semántica clara de actualización

Idea clave

“Sin ventana explícita” suele significar ventana global, no ausencia de tiempo y estado.

Tipo	Pregunta natural	Costo o riesgo
Fija	¿qué ocurrió en cada período?	bordes arbitrarios
Deslizante	¿qué ocurrió recientemente?	un evento se replica en varias ventanas
Sesión	¿qué actividad forma una interacción?	merge y estado por clave
Global	¿cuál es el estado acumulado?	crecimiento sin límite

Primero define la pregunta; después configura el tamaño.

Ventanas de 5 minutos

Un pico entre 10:04 y 10:06 queda partido entre dos ventanas.

Vista deslizante

Una ventana móvil puede capturar el pico completo, pero a mayor costo.

Preguntas de revisión

- ¿Los límites tienen significado para el negocio?
- ¿Necesitamos comparar períodos no solapados?
- ¿Qué anomalías podrían ocultarse en un borde?
- ¿La zona horaria y el horario de verano importan?

Caso	Ventana	Justificación
Ventas por día contable	fija	períodos comparables
CPU media de los últimos 10 min cada minuto	deslizante	horizonte móvil
Embudo de navegación por usuario	sesión	actividad delimitada por inactividad
Máximo histórico por sensor	global	estado acumulado

Para cada caso: especifica tamaño, período o gap y el timestamp que usarías.

Watermarks: progreso en tiempo de evento

Ventana

Retraso medio por ruta en $[10:00, 10:05)$.

A las 10:05 del reloj

Puede haber eventos aún en dispositivos, red, buffers o reintentos.

Esperar para siempre

Evita perder datos, pero nunca produce un resultado utilizable.

Idea clave

Necesitamos una estimación de cuánto ha avanzado la entrada en tiempo de evento.

Watermark = 10:05

El sistema cree haber observado todos —o casi todos— los eventos con timestamp anterior a 10:05.

No significa

- que el reloj de pared sea 10:05
- que el pipeline no tenga trabajo pendiente posterior
- que jamás aparecerá un evento tardío si la estimación es heurística

Partición	P0	P1	P2
Progreso	10:08	10:04	10:07

Watermark global aproximado = $\min(10:08, 10:04, 10:07) = 10:04$

Ventaja

Evita declarar completa una ventana mientras una partición conocida sigue atrás.

Costo

Una partición ociosa, lenta o detenida puede frenar el progreso global.

Propiedad

Si el watermark pasa t , no llegará ningún evento futuro con tiempo de evento menor que t .

Posible cuando

- la entrada es acotada
- la fuente ofrece progreso exacto
- existe orden y control completos

Consecuencia

No hay datos tardíos respecto a esa marca de agua; el estado puede cerrarse con certeza.

En fuentes distribuidas y abiertas, este conocimiento suele ser impráctico.

Señales posibles

- timestamps y orden dentro de particiones
- offsets observados y backlog
- particiones activas e inactivas
- patrones históricos de retraso
- metadatos de la fuente

Puede ser demasiado lento

Retrasa resultados aunque la mayoría de los datos ya llegó.

Puede ser demasiado rápido

Declara progreso y luego aparece un evento tardío.

Idea clave

El watermark ayuda a razonar sobre completitud, pero no resuelve por sí solo latencia y corrección.

Clase	Relación con watermark	Interpretación
Temprano	antes del cierre esperado	contribuye a una vista provisional
A tiempo	antes de que la marca pase el fin	incluido en la salida on-time
Tardío	llega después de que la marca pasó el fin	requiere política de corrección

Importante

“Tardío” no significa inválido ni duplicado. Describe cuándo se observó el evento respecto al progreso estimado.

Watermark

Estima progreso en **tiempo de evento** y habilita decisiones por ventana.

SLA de latencia

Compromete cuánto demora el sistema en producir o actualizar un resultado.

Idea clave

Podemos tener un watermark preciso pero lento, o un pipeline rápido con resultados incompletos.

Estado inicial

Ventana [10:00, 10:05) · watermark 10:03 · eventos vistos: 10:00:30, 10:02:10, 10:04:45.

Luego ocurre

1. El watermark avanza a 10:06.
2. Llega un evento con `event_time = 10:01:20`.
3. Llega otro con `event_time = 10:06:10`.

- ¿Cuál evento es tardío?
- ¿A qué ventana pertenece cada uno?
- ¿Qué política necesitamos para que el primero corrija el resultado?

Triggers: cuándo materializar

Ventana `[10:00, 10:05)`

Puede emitir más de una vez mientras llegan datos y avanza el watermark.

Temprano

valor 8



A tiempo

valor 11



Tardío

valor 12

Idea clave

Un pane no es otra ventana: es una emisión particular del resultado de la misma ventana.

Ejemplo

Emitir cada 30 segundos de tiempo de procesamiento antes de que el watermark alcance el fin de la ventana.

Beneficio

Dashboard y alertas obtienen una señal rápidamente.

Costo

El valor cambia; consumidores deben aceptar actualizaciones.

Una salida temprana debe etiquetarse como parcial o tener una clave que permita sobrescribirla.

Condición típica

Emitir cuando el watermark pasa el final de la ventana.

Lo que ofrece

Una versión basada en la mejor estimación disponible de completitud.

Lo que no promete

Ser inmutable si el watermark es heurístico y aceptamos datos tardíos.

On-time significa “según el watermark”, no “absolutamente final”.

Condición típica

Después de la emisión on-time, volver a emitir cuando llega cada evento tardío o tras acumular varios.

El consumidor debe saber

- qué ventana se actualiza
- si el nuevo valor reemplaza al anterior o es un delta
- si existe una versión o secuencia
- cuándo dejará de aceptar correcciones

Idea clave

La política de salida es un contrato entre el pipeline y el sistema de serving.

Señal	Ejemplo	Uso
Watermark	al pasar el fin	salida on-time
Processing time	cada 30 segundos	actualización temprana
Conteo	cada 1.000 eventos	control de volumen
Dato	evento flush	frontera del dominio
Composición	A o B; A y B	políticas híbridas

Señales descritas en [Streaming 102](#).

Modo	Pane 1	Pane 2	Lectura
Discarding	7	3 + 4	deltas independientes
Accumulating	7	14	nuevo total reemplaza
Retracting	7	14 y -7	corrección explícita

Idea clave

Sumar panes acumulativos vuelve a contar datos; reemplazarlos es la operación correcta.

Pipeline

Emite solo lo nuevo desde el pane anterior.

Consumidor

Suma los deltas de manera durable e idempotente.

Riesgo

Si se pierde un delta o se procesa dos veces, el total downstream queda incorrecto.

Necesita identidad de pane, orden o deduplicación: los conceptos de la clase 3 reaparecen.

Clave de salida

`(route_id, window_start, window_end)`

Valor

`avg_delay = 3.8 min, pane = LATE, version = 4, updated_at = ...`

Ventaja

La versión más reciente representa el total conocido.

Costo

El pipeline conserva estado y recomputa o actualiza el agregado.

Sesiones que se fusionan

Un evento tardío puede unir dos sesiones publicadas previamente en una sola ventana.

Antes

sesión A + sesión B



Después

retractar A y B; insertar C

Una simple sobrescritura no siempre identifica qué resultados anteriores deben desaparecer.

Política

- **Early:** cada 30 s de processing time.
- **On-time:** cuando el watermark pasa el fin.
- **Late:** por cada evento tardío aceptado.
- **Accumulation:** total acumulado que reemplaza la versión anterior.

Idea clave

La política produce baja latencia, una señal de completitud y correcciones posteriores, a cambio de más estado y escrituras.

Datos tardíos y cierre

Ejemplo

Ventana termina a 10:05 · allowed lateness = 10 min.

Watermark	Estado	Evento tardío
10:12	ventana retenida	puede actualizar
10:16	horizonte superado	se descarta o desvía

En Beam, el estado puede recolectarse cuando el watermark supera fin de ventana + lateness permitido.

Aceptar

Actualizar el resultado mientras la ventana siga viva.

Desviar

Enviar a un tópico o tabla de late data para análisis.

Descartar

Registrar métrica y eliminar cuando el valor no justifica el costo.

Idea clave

Descartar puede ser una decisión correcta si se mide, se justifica y el impacto es tolerable.

Horizonte	Completitud	Estado	Actualizaciones
Corto	menor	menor	pocas
Largo	mayor	mayor	más correcciones
Infinito	máxima teórica	no acotado	contrato nunca final

La política debe apoyarse en la distribución real de retrasos.

Métricas útiles

- `event_lag = ingest_time - event_time`
- percentiles p50, p95, p99 y máximo
- fracción tardía por fuente o partición
- eventos descartados por horizonte
- watermark lag
- número de panes y actualizaciones por ventana

Idea clave

Un promedio de 2 segundos puede ocultar una cola de eventos que tarda horas.

```
{  
  "route_id": "L30",  
  "window": {"start": "10:00", "end": "10:05"},  
  "avg_delay_min": 3.8,  
  "pane_timing": "LATE",  
  "pane_index": 3,  
  "is_final": false  
}
```

Discusión

¿Qué significa `is_final` si una corrección administrativa puede llegar mañana? La frontera debe ser explícita.

Latencia

resultados tempranos y frecuentes

Complejidad

watermark + lateness amplio

Costo

estado, cómputo y escrituras

No existe una configuración universal

Un dashboard exploratorio, una factura y una alerta de fraude necesitan compromisos distintos.



10 minutos

Al volver: diseñamos una política completa para eventos de movilidad.

Diseño de una política completa

Requisitos

- eventos `trip_update` con timestamp del vehículo
- métrica operativa visible en menos de 45 segundos
- correcciones durante 15 minutos
- p99 del retraso de llegada observado: 8 minutos
- destino con upsert por clave

Diseña: reloj · ventana · trigger · lateness · acumulación

Elección

Usar `event_time` de la observación GPS validado contra límites plausibles.

Por qué

El retraso debe atribuirse al período del viaje, no al momento en que volvió la conectividad.

Protección

Eventos con reloj imposible —demasiado futuro o demasiado antiguo— se desvían para revisión.

Configuración

```
[10:00, 10:05), [10:05, 10:10),  
...
```

Clave de agrupación

```
(route_id, window_start)
```

Justificación

Períodos cortos y comparables para operación; una ventana deslizante costaría más y no es necesaria para este tablero.

Pane	Trigger	Objetivo
Early	cada 30 s de processing time	cumplir latencia
On-time	watermark pasa fin	señal de completitud
Late	por evento tardío	corregir rápidamente

Idea clave

La ventana es de cinco minutos, pero eso no obliga a esperar cinco minutos para mostrar el primer valor.

Modo

Cada pane contiene el total acumulado conocido para la ventana.

Destino

Upsert por (route_id, window_start, window_end) con pane_index creciente.

Efecto

El tablero reemplaza 3,2 min por 3,6 min; no suma ambos valores.

Motivo

Supera el p99 observado de 8 min y deja margen para incidentes menores.

Fuera del horizonte

Enviar a `trip_updates.too_late` y medir por origen.

Idea clave

La elección se revisa con datos operativos; no es una constante eterna.

```
updates = (  
  events  
  | "AssignEventTime" >> beam.Map(with_device_timestamp)  
  | "FiveMinutes" >> beam.WindowInto(  
    FixedWindows(5 * 60),  
    trigger=AfterWatermark(  
      early=Repeatedly(AfterProcessingTime(30)),  
      late=AfterCount(1)),  
    accumulation_mode=AccumulationMode.ACCUMULATING,  
    allowed_lateness=15 * 60)  
  | "AverageByRoute" >> beam.CombinePerKey(MeanCombineFn())  
)
```

API ilustrativa basada en [Apache Beam Programming Guide](#) se implementará con detalle en clases 5–6.

Casos mínimos

- eventos en ambos lados de un borde de ventana
- llegada desordenada dentro de la misma ventana
- evento tardío dentro de los 15 minutos
- evento demasiado tardío
- watermark detenido por una partición
- replay del mismo conjunto de eventos
- panes repetidos o fuera de orden en el destino

Idea clave

Una configuración de tiempo se valida con secuencias controladas, no solo mirando un dashboard en vivo.

Significado

- timestamp y zona horaria
- tipo y tamaño de ventana
- interpretación de bordes
- semántica de cada pane

Operación

- fuente del watermark
- horizonte de lateness
- crecimiento del estado
- salida de eventos
descartados
- métricas de lag y
correcciones

Síntesis y aplicación

Consigna

Analizar una secuencia de eventos desordenados o tardíos y proponer una política temporal.

Debe incluir

- elección y justificación del tiempo de evento
- tipo y configuración de ventana
- política de watermark y lateness
- interpretación de resultados parciales
- modo de acumulación y contrato de salida
- discusión del trade-off latencia–completitud–costo

Modalidad individual · Peso 15%

Evidencia sugerida

1. Tabla pequeña con tiempo de evento y tiempo de llegada.
2. Ventanas asignadas a cada evento.
3. Watermark o regla de progreso simulada.
4. Secuencia de panes: early, on-time y late.
5. Resultado final o evento descartado.
6. Explicación de qué cambiaría con otra política.

No se requiere programación: se evalúa el razonamiento sobre tiempo, ventanas y resultados.

Conceptuales

- confundir ventana con trigger
- llamar “final” al pane on-time
- tratar tardío como duplicado
- usar tiempo de llegada sin justificar

Operativos

- estado sin expiración
- panes sin clave o versión
- descartar sin métricas
- ignorar particiones ociosas

Pregunta	Mecanismo	Decisión visible
Qué	transformación	métrica o modelo
Dónde	ventana	frontera temporal
Cuándo	watermark + trigger	momento de publicación
Cómo	acumulación	delta, reemplazo o retractación

Idea clave

Si falta una respuesta, el consumidor tendrá que adivinar el significado del resultado.

Hoy

- El tiempo de evento preserva cuándo ocurrió el hecho; el de procesamiento describe cuándo lo observamos.
- Las ventanas hacen finitas las agregaciones, pero no deciden cuándo emitir.
- El watermark estima progreso y puede equivocarse.
- Los triggers producen panes tempranos, on-time y tardíos.
- Lateness y acumulación convierten correcciones en un contrato explícito con costo medible.

Bibliografía base

- Tyler Akidau — [The world beyond batch: Streaming 101](#).
- Tyler Akidau — [The world beyond batch: Streaming 102](#).
- Tyler Akidau, Slava Chernyak y Reuven Lax — **Streaming Systems**, capítulos sobre tiempo y ventanas.
- Apache Beam — [Programming Guide: Windowing](#).
- Apache Flink — [Timely Stream Processing](#).

Hasta ahora

Diseñamos el significado temporal de un pipeline: ventanas, emisiones y correcciones.

Próxima pregunta

¿Cómo expresamos estas transformaciones como un pipeline ejecutable, portable y comprobable?

Apache Beam/Python · Pipeline · PCollection · PTransform

- Objetivos, temas, resultados esperados y Tarea 2: syllabus de la asignatura.
- Dominios de tiempo, datos no acotados y estrategias de ventana: Tyler Akidau, [Streaming 101](#).
- Marco qué/dónde/cuándo/cómo, watermarks, triggers, panes y acumulación: Tyler Akidau, [Streaming 102](#).
- Ventanas, triggers, allowed lateness y recolección de estado: [Apache Beam Programming Guide](#).
- Progreso temporal y watermarks en sistemas distribuidos: [Apache Flink — Timely Stream Processing](#).
- Figuras de dominios de tiempo y estrategias de ventana: Tyler Akidau; atribuciones registradas en `images/ATTRIBUTIONS.md`.

Uso didáctico

Los ejemplos de movilidad y las tablas de eventos fueron elaborados para esta clase.

¿Preguntas?

 rmparra@pol.una.py

 **Bibliografía base: Streaming Systems + Streaming 101/102**

 **Próxima clase: Apache Beam/Python**